

```

import asyncio
import aiohttp
import logging
import os
import json
from datetime import datetime, timedelta
import asyncpg

from database import (
    init_db, get_risk_state, get_dynamic_limits, reset_loss_streak,
    log_alert, log_opportunity, update_price_history, get_price_history,
    log_sentiment, get_daily_stats, get_alerted_markets,
    get_logged_opportunities, check_resolutions,
    update_open_positions, run_weekly_backtest,
    record_alert_snapshot, cleanup_old_snapshots
)
from scoring import score_opportunity, is_market_active, detect_category
from research import get_fear_greed, build_research_summary, get_crypto_data, pre
from analysis import (
    analyze_price_momentum, analyze_price_velocity,
    analyze_liquidity, check_cross_market_consistency,
    detect_polymarket_lag, analyze_event_timing,
    check_resolution_ambiguity, calculate_confidence_tier
)
from telegram import (
    send_message, send_alert, get_updates, answer_callback,
    send_status, send_daily_summary, send_heartbeat, send_weekly_analysis
)

CONFIG = {
    "check_interval_seconds": 30,
    "min_score_for_alert": 85,
    "log_opportunity_threshold": 40,
    "markets_per_page": 100,
    "max_pages": 50,
    "summary_hour_utc": 13,
    "heartbeat_hour_utc": 12,
    "resolution_check_hours": [6, 8, 10, 12, 14, 16, 18, 20, 22],
    "weekly_analysis_day": 6,
}

TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN")
TELEGRAM_CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID")
DATABASE_URL = os.environ.get("DATABASE_URL")

```

```

ODDS_API_KEY = os.environ.get("ODDS_API_KEY")

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(message)s",
    datefmt="%H:%M:%S"
)
log = logging.getLogger(__name__)

def now():
    return datetime.utcnow()

async def fetch_all_markets():
    all_markets = []
    offset = 0
    headers = {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.",
        "Accept": "application/json"
    }
    async with aiohttp.ClientSession() as session:
        while offset < CONFIG["max_pages"] * CONFIG["markets_per_page"]:
            url = (
                "https://gamma-api.polymarket.com/markets"
                "?active=true&closed=false"
                "&limit=" + str(CONFIG["markets_per_page"])
                + "&offset=" + str(offset)
                + "&order=volume24hr&ascending=false"
            )
            try:
                async with session.get(url, headers=headers,
                                       timeout=aiohttp.ClientTimeout(total=15)) as resp:
                    if resp.status == 200:
                        markets = await resp.json()
                        if not markets:
                            break
                        all_markets.extend(markets)
                        page = offset // CONFIG["markets_per_page"] + 1
                        log.info("Page %d: %d markets (total: %d)",
                               page, len(markets), len(all_markets))
                        if len(markets) < CONFIG["markets_per_page"]:
                            break
                        offset += CONFIG["markets_per_page"]
                        await asyncio.sleep(0.5)
                    else:
                        log.error("API error: %d", resp.status)
                        break
            except Exception as e:

```

```

        log.error("Fetch error: %s", e)
        break
    return all_markets

async def load_upcoming_events(conn):
    try:
        rows = await conn.fetch("""
            SELECT event_name, event_date, category, relevance_keywords
            FROM event_calendar
            WHERE event_date > NOW()
            ORDER BY event_date ASC
        """)
        return [dict(r) for r in rows]
    except Exception as e:
        log.warning("load_upcoming_events error: %s", e)
        return []

async def seed_event_calendar(conn):
    """Seed known upcoming events into the calendar."""
    events = [
        ("FOMC Meeting", "2026-03-18 18:00:00", "Economics", "fed,federal reserve"),
        ("NBA Playoffs Start", "2026-04-15 00:00:00", "Sports", "nba,playoffs,bas"),
        ("MLB Season Start", "2026-03-26 00:00:00", "Sports", "mlb,baseball,world"),
        ("US CPI Report", "2026-03-12 12:30:00", "Economics", "cpi,inflation,cons"),
        ("US Jobs Report", "2026-04-03 12:30:00", "Economics", "jobs,unemployment"),
        ("FIFA World Cup 2026", "2026-06-11 00:00:00", "Sports", "world cup,fifa,"),
        ("US Midterms", "2026-11-03 00:00:00", "Politics", "election,senate,congr")
    ]
    for name, date_str, category, keywords in events:
        existing = await conn.fetchrow(
            "SELECT id FROM event_calendar WHERE event_name=$1", name
        )
        if not existing:
            await conn.execute("""
                INSERT INTO event_calendar (event_name, event_date, category, rel
                VALUES ($1, $2, $3, $4, $5)
                """, name, datetime.fromisoformat(date_str), category, keywords, now())
            log.info("Event calendar seeded")

async def process_updates(conn, updates, last_update_id):
    new_last_id = last_update_id
    for update in updates:
        update_id = update.get("update_id", 0)
        if update_id <= last_update_id:
            continue
        new_last_id = max(new_last_id, update_id)

```

```

message = update.get("message", {})
if message:
    text = message.get("text", "")
    if text and "/status" in text.lower():
        log.info("Status command received")
        await send_status(TELEGRAM_TOKEN, TELEGRAM_CHAT_ID, conn)

callback = update.get("callback_query")
if callback:
    data = callback.get("data", "")
    callback_id = callback.get("id")
    await answer_callback(TELEGRAM_TOKEN, callback_id)
    if data.startswith("agree_") or data.startswith("disagree_"):
        parts = data.split("_")
        rating = parts[0]
        alert_id = parts[1]
        await conn.execute(
            "UPDATE alerts SET user_rating=$1 WHERE id=$2",
            rating, int(alert_id)
        )
        emoji = "👍" if rating == "agree" else "👎"
        await send_message(
            TELEGRAM_TOKEN, TELEGRAM_CHAT_ID,
            emoji + " Feedback recorded for alert #" + alert_id + "\n"
            "Rating: " + rating.capitalize()
        )
    return new_last_id

async def main():
    log.info("=" * 50)
    log.info("Polymarket Bot v15 Starting...")
    log.info("Multi-file architecture")
    log.info("Fixed: profitability logic")
    log.info("Fixed: category double-tagging")
    log.info("Fixed: loss streak reset")
    log.info("New: price momentum detection")
    log.info("New: price velocity alerts")
    log.info("New: liquidity depth checking")
    log.info("New: cross-market consistency")
    log.info("New: polymarket lag detection")
    log.info("New: event timing awareness")
    log.info("New: resolution ambiguity detection")
    log.info("New: Economics and Science categories")
    log.info("New: confidence tiers HIGH/MEDIUM/LOW")
    log.info("=" * 50)

conn = await asyncpg.connect(DATABASE_URL)

```

```

await init_db(conn)
await seed_event_calendar(conn)
await reset_loss_streak(conn)

await send_message(
    TELEGRAM_TOKEN, TELEGRAM_CHAT_ID,
    "<b>Polymarket Bot v15 Started!</b>\n\n"
    "Multi-file professional architecture\n\n"
    "Fixed:\n"
    "Profitability logic corrected\n"
    "Category double-tagging fixed\n"
    "False loss streak reset\n\n"
    "New:\n"
    "Price momentum detection\n"
    "Price velocity alerts\n"
    "Liquidity depth checking\n"
    "Cross-market consistency\n"
    "Polymarket lag detection\n"
    "Event timing awareness\n"
    "Resolution ambiguity warnings\n"
    "Economics and Science categories\n"
    "Confidence tiers HIGH MEDIUM LOW\n\n"
    "Type /status to check stats!"
)

alerted_markets = await get_alerted_markets(conn)
logged_opportunities = await get_logged_opportunities(conn)
last_summary_date = None
last_heartbeat_date = None
last_weekly_date = None
last_resolution_hours = set()
last_update_id = 0
consecutive_errors = 0
fear_greed_cache = {"data": None, "cached_at": None}

while True:
    try:
        current_time = now()

        # Process Telegram updates
        updates = await get_updates(TELEGRAM_TOKEN, last_update_id + 1)
        if updates:
            last_update_id = await process_updates(conn, updates, last_update

        # Refresh Fear and Greed every hour
        if (fear_greed_cache["cached_at"] is None or
            (now() - fear_greed_cache["cached_at"]).total_seconds() > 360

```

```

    fear_greed = await get_fear_greed()
    fear_greed_cache["data"] = fear_greed
    fear_greed_cache["cached_at"] = now()
    if fear_greed.get("success"):
        await log_sentiment(conn, fear_greed)
        log.info("Fear and Greed: %d (%s) %s",
                fear_greed["score"], fear_greed["regime"],
                fear_greed.get("trend", ""))
    else:
        fear_greed = fear_greed_cache["data"]

# Scheduled tasks
if (current_time.hour == CONFIG["heartbeat_hour_utc"] and
    current_time.date() != last_heartbeat_date):
    await send_heartbeat(TELEGRAM_TOKEN, TELEGRAM_CHAT_ID, conn)
    last_heartbeat_date = current_time.date()

if (current_time.hour == CONFIG["summary_hour_utc"] and
    current_time.date() != last_summary_date):
    await send_daily_summary(TELEGRAM_TOKEN, TELEGRAM_CHAT_ID, conn)
    last_summary_date = current_time.date()

if (current_time.weekday() == CONFIG["weekly_analysis_day"] and
    current_time.date() != last_weekly_date):
    await send_weekly_analysis(TELEGRAM_TOKEN, TELEGRAM_CHAT_ID, conn)
    backtest_report = await run_weekly_backtest(conn)
    await send_message(TELEGRAM_TOKEN, TELEGRAM_CHAT_ID, backtest_rep
    await cleanup_old_snapshots(conn)
    last_weekly_date = current_time.date()

resolution_key = str(current_time.date()) + "_" + str(current_time.ho
if (current_time.hour in CONFIG["resolution_check_hours"] and
    resolution_key not in last_resolution_hours):
    log.info("Running resolution check...")
    resolved_count = await check_resolutions(conn)
    last_resolution_hours.add(resolution_key)
    if len(last_resolution_hours) > 100:
        last_resolution_hours = set(list(last_resolution_hours)[-50:])
    if resolved_count > 0:
        await send_message(
            TELEGRAM_TOKEN, TELEGRAM_CHAT_ID,
            "<b>Resolution Check Complete</b>\n\n"
            "Resolved " + str(resolved_count) + " markets\n"
            "Type /status to see updated win rate!"
        )

# Monitor open positions for take-profit / stop-loss

```

```

closed_positions = await update_open_positions(conn)
for pos in closed_positions:
    emoji = "✅" if pos["profitable"] else "❌"
    direction = "+" if pos["return_pct"] >= 0 else ""
    msg = (
        f"{emoji} <b>Position Closed [{pos['outcome_type']}]</b>\n\n"
        f"{pos['question'][:80]}\n\n"
        f"Entry: {round(pos['entry_price']*100)}¢ → "
        f"Exit: {round(pos['exit_price']*100)}¢\n"
        f"Return: {direction}{pos['return_pct']}% "
        f"(Peak was +{pos['peak_return_pct']}%)\n"
        f"Reason: {pos['exit_reason']}"
    )
    await send_message(TELEGRAM_TOKEN, TELEGRAM_CHAT_ID, msg)

# Load upcoming events for this scan
upcoming_events = await load_upcoming_events(conn)

# Main market scan
# Pre-fetch all coin prices in one request before scanning
await prefetch_all_crypto()

log.info("Scanning Polymarket markets...")
markets = await fetch_all_markets()

if not markets:
    consecutive_errors += 1
    log.warning("No markets returned (error streak: %d)", consecutive_errors)
    if consecutive_errors >= 10:
        await send_message(
            TELEGRAM_TOKEN, TELEGRAM_CHAT_ID,
            "<b>Bot Error Alert</b>\n\n"
            "Could not reach Polymarket API for 5+ minutes\n"
            "Bot is still running and retrying"
        )
        consecutive_errors = 0
else:
    consecutive_errors = 0
    active_markets = [m for m in markets if is_market_active(m)]
    log.info("%d total -> %d active after filtering",
             len(markets), len(active_markets))

    state = await get_risk_state(conn)
    limits = get_dynamic_limits(state)

    opportunities = []

```

```

for market in active_markets:
    question_lower = market.get("question", "").lower()

    # Skip low-edge categories
    esports_keywords = [
        "counter-strike", "cs2", "valorant", "dota", "league of l
        "lol:", "bo3", "bo5", "esport", "gaming league", "blast",
        "esl", "faceit", "dreamhack", "iem ", "majors:"
    ]
    weather_keywords = [
        "temperature", "rainfall", "precipitation", "hurricane",
        "tornado", "snowfall", "weather", "degrees", "fahrenheit"
        "celsius", "highest temp", "lowest temp", "wind speed"
    ]
    tennis_keywords = [
        "atp ", "wta ", "open tennis", "wimbledon",
        "french open", "us open tennis", "roland garros"
    ]

    if any(kw in question_lower for kw in esports_keywords):
        continue
    if any(kw in question_lower for kw in weather_keywords):
        continue
    if any(kw in question_lower for kw in tennis_keywords):
        continue

    result = score_opportunity(
        market,
        all_markets=active_markets,
        upcoming_events=upcoming_events,
        fear_greed=fear_greed
    )
    score = result["score"]
    reason = result["reason"]
    category = result["category"]
    market_age = result["signals"]["age_hours"]

    if score < CONFIG["log_opportunity_threshold"]:
        continue

    question = market.get("question", "Unknown")
    outcomes = market.get("outcomePrices", "[0.5]")
    if isinstance(outcomes, str):
        outcomes = json.loads(outcomes)
    yes_price = float(outcomes[0]) if outcomes else 0.5
    market_id = str(market.get("id", question[:50]))
    volume = float(market.get("volumeNum", 0) or 0)

```



```

# — Rich context data collection —————
# Days to resolution
days_to_resolution = None
end_date_raw = market.get("endDate") or market.get("end_date")
if end_date_raw:
    try:
        from datetime import timezone
        end_dt = datetime.fromisoformat(
            str(end_date_raw).replace("Z", "+00:00")
        ).replace(tzinfo=None)
        days_to_resolution = round(
            (end_dt - now()).total_seconds() / 86400, 1
        )
    except Exception:
        pass

# Bid/ask spread from market data
bid_price = float(market.get("bestBid", 0) or 0) or None
ask_price = float(market.get("bestAsk", 0) or 0) or None

# Score breakdown for backtest analysis
score_breakdown = {
    "base": 50,
    "liquidity": result["signals"].get("liquidity", {}).get("si
    "momentum": result["signals"].get("momentum", {}).get("si
    "velocity": result["signals"].get("velocity", {}).get("fa
    "ambiguity": bool(result["signals"].get("ambiguity")),
    "lag": bool(result["signals"].get("lag")),
    "age_hours": market_age,
    "final_score": score,
}

opp = {
    "id": market_id,
    "question": question,
    "score": score,
    "reason": reason,
    "yes_price": yes_price,
    "volume": volume,
    "age": market_age,
    "category": category,
    "confidence_tier": "MEDIUM",
    "days_to_resolution": days_to_resolution,
    "bid_price": bid_price,
    "ask_price": ask_price,
    "score_breakdown": score_breakdown,

```

```

}

# Run analysis modules
liquidity = analyze_liquidity(market)
if not liquidity["liquid"]:
    opp["liquidity_warning"] = liquidity["warning"]

ambiguity = check_resolution_ambiguity(question)
if ambiguity:
    opp["ambiguity_warning"] = ambiguity

event_matches = analyze_event_timing(market, upcoming_events)
if event_matches:
    opp["upcoming_events"] = event_matches

inconsistencies = check_cross_market_consistency(market, acti
if inconsistencies:
    opp["inconsistencies"] = inconsistencies

# Price history based analysis
confirming = 0
contradicting = 0

if market_id in alerted_markets:
    await update_price_history(conn, market_id, yes_price)
    history = await get_price_history(conn, market_id)

    if history:
        momentum = analyze_price_momentum(history)
        opp["momentum"] = momentum
        if momentum["signal"] in ["RISING", "STRONG_RISING"]:
            confirming += 1
        elif momentum["signal"] in ["FALLING", "STRONG_FALLIN
            contradicting += 1

        velocity = analyze_price_velocity(history)
        if velocity["fast_move"]:
            opp["velocity_alert"] = velocity["alert"]
            confirming += 1

# Extract price context: what was price 1d/3d/7d ago
try:
    now_ts = now()
    def price_n_days_ago(h, days):
        cutoff = now_ts - timedelta(hours=days*24)
        candidates = [r for r in h if r["recorded_at"
        return float(candidates[0]["yes_price"]) if c

```

```

        opp["price_1d_ago"] = price_n_days_ago(history, 1
        opp["price_3d_ago"] = price_n_days_ago(history, 3
        opp["price_7d_ago"] = price_n_days_ago(history, 7
    except Exception:
        pass

    # Record dense snapshot for post-alert price curve
    alert_row = await conn.fetchrow(
        "SELECT id, alerted_at FROM alerts WHERE market_id=$1
        market_id
    )
    if alert_row:
        await record_alert_snapshot(
            conn, alert_row["id"], market_id, yes_price,
            alert_row["alerted_at"],
            bid_price=opp.get("bid_price"),
            ask_price=opp.get("ask_price")
        )

    # Polymarket lag detection for crypto
    if category == "Crypto":
        crypto_data = await get_crypto_data(question)
        lag = detect_polymarket_lag(question, yes_price, crypto_d
        if lag:
            opp["lag_detected"] = lag
            confirming += 2

    # Calculate confidence tier
    opp["confidence_tier"] = calculate_confidence_tier(
        score, confirming, contradicting
    )

    # Build signals_fired string for backtest tracking
    fired = []
    if opp.get("momentum", {}).get("signal") in ["RISING", "STRON
        fired.append("momentum_up")
    if opp.get("momentum", {}).get("signal") in ["FALLING", "STRO
        fired.append("momentum_down")
    if opp.get("velocity_alert"):
        fired.append("velocity")
    if opp.get("lag_detected"):
        fired.append("lag_detected")
    if opp.get("inconsistencies"):
        fired.append("cross_market")
    if opp.get("upcoming_events"):
        fired.append("event_timing")
    if opp.get("liquidity_warning"):

```

```

        fired.append("low_liquidity")
    if opp.get("ambiguity_warning"):
        fired.append("ambiguous")
    opp["signals_fired"] = ",".join(fired)

    # Log all 40+ opportunities silently
    if market_id not in logged_opportunities:
        await log_opportunity(conn, opp, fear_greed, market_age)
        logged_opportunities.add(market_id)

    # Only alert on 70+
    if score >= CONFIG["min_score_for_alert"]:
        opportunities.append(opp)

opportunities.sort(key=lambda x: x["score"], reverse=True)

if opportunities:
    log.info("Found %d alertable opportunities", len(opportunities))
    for opp in opportunities[:5]:
        log.info("Score:%d [%s] %s",
                opp["score"], opp["category"], opp["question"][:
        if opp["id"] not in alerted_markets:
            alert_id = await log_alert(
                conn, opp, fear_greed, opp.get("age")
            )
            alerted_markets.add(opp["id"])

            research = await build_research_summary(
                opp["question"], opp["yes_price"],
                opp["category"], fear_greed, ODDS_API_KEY
            )

            await send_alert(
                TELEGRAM_TOKEN, TELEGRAM_CHAT_ID,
                opp, alert_id, research, limits
            )

        else:
            log.info("No new alertable opportunities this scan")

except Exception as e:
    log.error("Unexpected error: %s", e)
    await send_message(
        TELEGRAM_TOKEN, TELEGRAM_CHAT_ID,
        "<b>Bot Error Alert</b>\n\n"
        "Unexpected error: " + str(e)[:200] + "\n"
        "Bot is attempting to recover automatically"

```

```
)

log.info("Next scan in %d seconds", CONFIG["check_interval_seconds"])
log.info("-" * 50)
await asyncio.sleep(CONFIG["check_interval_seconds"])

if __name__ == "__main__":
    asyncio.run(main())
```